ISEA SUMMER INTERNSHIP 2026

Secure Network Application Development Using TCP

Assignment 7 Report

SentinelChat — Secure Multi-Client TCP Chat Platform

Submitted By: Khatija Fathima

Roll No.: 323506402225

Course: ISEA Summer Internship 2026

Date: 16 July 2026

Table of Contents

1. Objective

The objective of this assignment is to enhance the multi-client TCP chat application developed in Assignment 6 by implementing practical security mechanisms. The application focuses on secure authentication, password protection using SHA-256 hashing, duplicate login prevention, failed login protection, session timeout management, secure logging, input validation, and verification of network communication using Wireshark.

The project demonstrates how application-level security can be integrated into a TCP-based communication system while maintaining usability and a modern graphical interface.

2. Introduction

Modern communication applications must provide secure authentication and protect user information from unauthorized access. A simple TCP chat application is functional, but without security mechanisms it becomes vulnerable to password theft, session hijacking, duplicate logins, and unauthorized access.

SentinelChat was developed as a secure multi-client chat platform that combines TCP socket programming with practical security mechanisms. Instead of focusing on complex cryptography, this project emphasizes real-world application security techniques that are commonly used in secure software systems.

The application provides a modern graphical user interface and supports multiple users communicating simultaneously through private messaging, broadcast messaging, group chat, chat history management, server monitoring, and secure authentication. The dashboard below shows the main control centre reached immediately after a successful login.

Fig 2.1 — SentinelChat Dashboard (Cyber Operations Center)

3. Security Features Implemented

3.1 User Authentication

Username and password authentication was implemented before allowing any user to access the application. Only registered users stored inside users.json are allowed to log in. The Server IP, username, and password fields are validated by the server before a session is created.

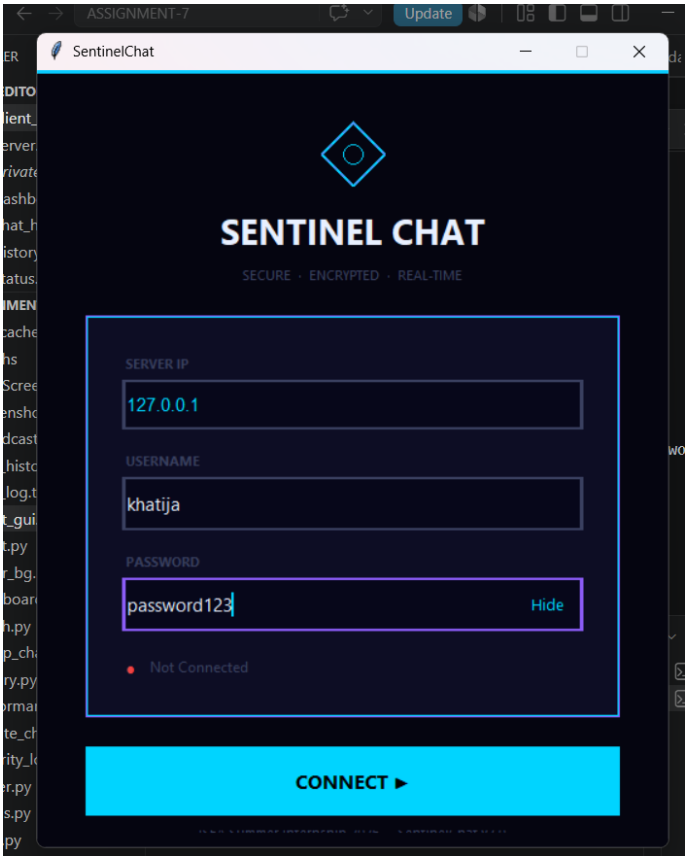


Fig 3.1 — SentinelChat Login Window

3.2 Secure Password Storage

Passwords are never stored in plaintext. Every password is converted into a SHA-256 hash using Python's hashlib library before storage. This ensures that even if the user database is compromised, the original passwords cannot be directly recovered.

```
users.json
{
    "users": {
        "admin":    { "password": "ef92b778bafe771e89245b89ecbc08a4...94f" },
        "khatija":  { "password": "ef92b778bafe771e89245b89ecbc08a4...94f" },
        "fathima":  { "password": "ef92b778bafe771e89245b89ecbc08a4...94f" },
        "sowmya":   { "password": "ef92b778bafe771e89245b89ecbc08a4...94f" },
        "sreekari": { "password": "ef92b778bafe771e89245b89ecbc08a4...94f" }
    }
}
```

Each stored value is a 256-bit SHA-256 digest rather than the original text password, confirming that plaintext credentials are never written to disk.

3.3 Duplicate Login Prevention

The server maintains a list of currently logged-in users. If the same account attempts to log in from another client while already active, the login request is rejected. This prevents account sharing and session hijacking.

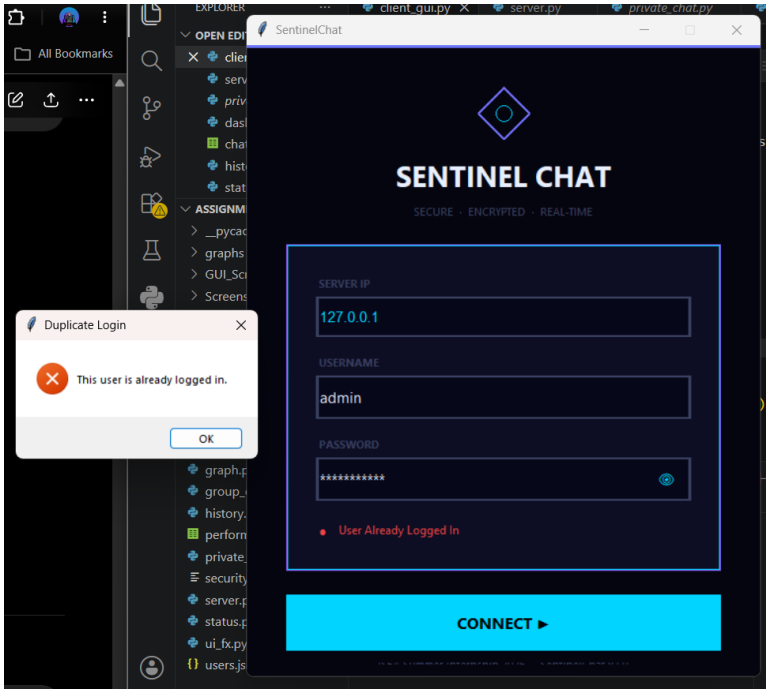


Fig 3.2 — Duplicate Login Rejected by Server

3.4 Failed Login Protection

To protect against brute-force attacks, the application temporarily locks an account after five consecutive failed login attempts. A countdown timer informs the user when the account will become available again.

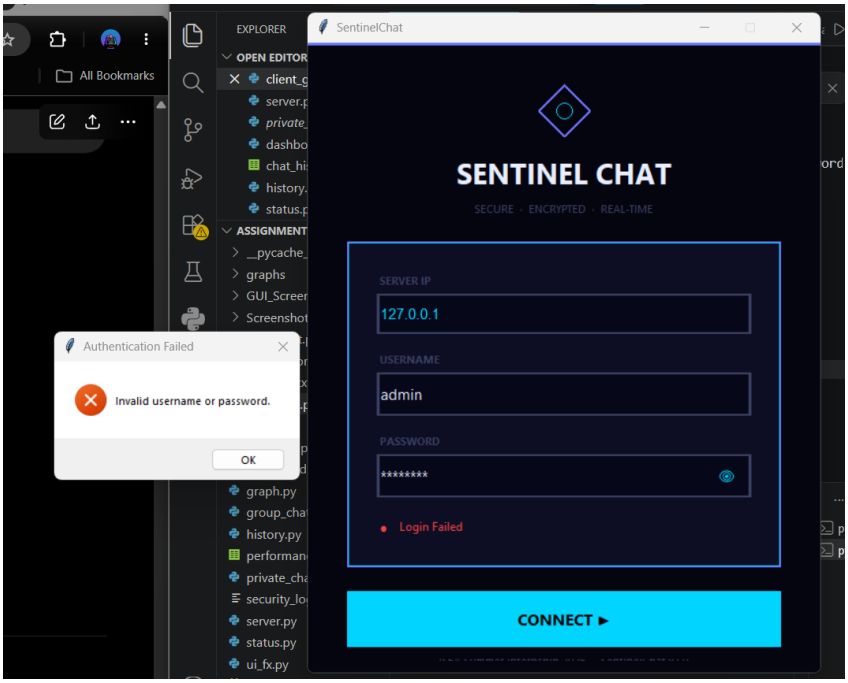


Fig 3.3 — Invalid Password Rejected

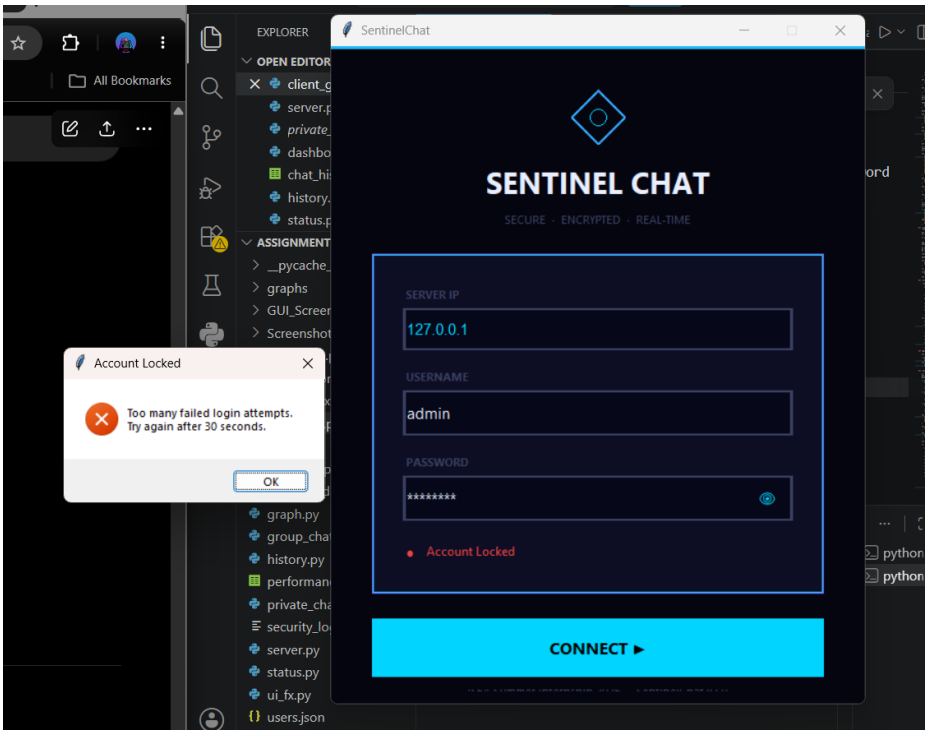


Fig 3.4 — Account Locked After Repeated Failed Attempts

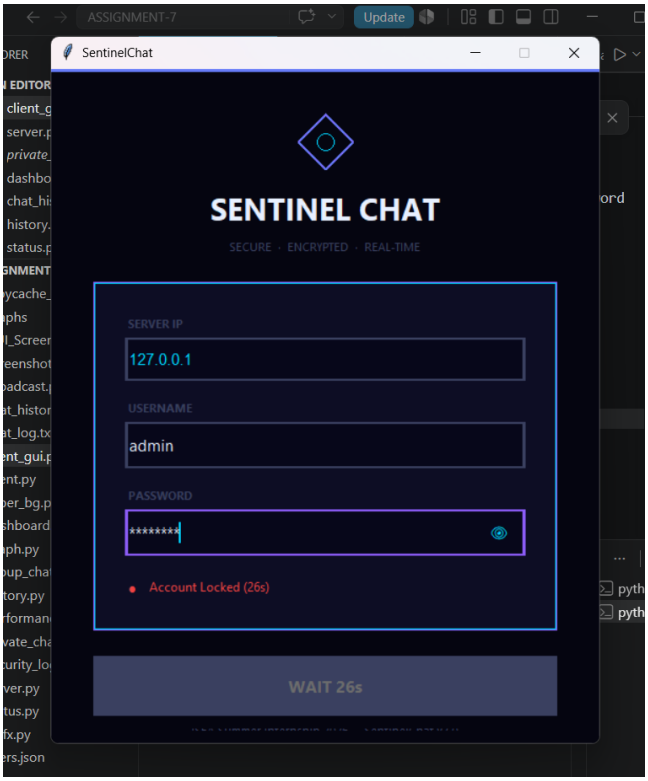


Fig 3.5 — Lockout Countdown Timer

3.5 Session Timeout

User activity is continuously monitored. If no activity occurs for three minutes, the session automatically expires. The dashboard closes, the socket connection is terminated, and the login screen is displayed again. This feature prevents unauthorized access when a user leaves the system unattended.

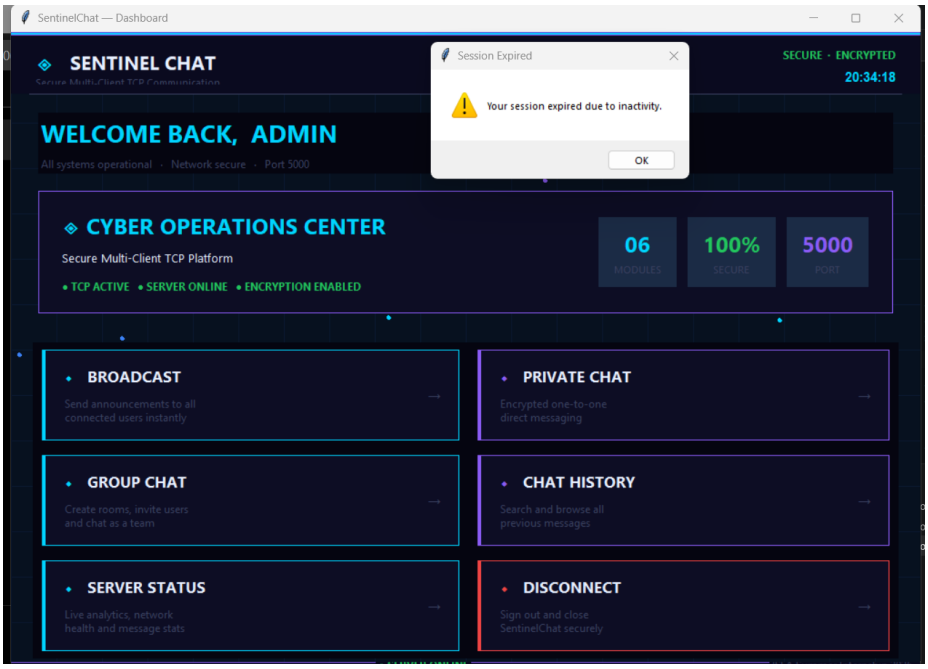


Fig 3.6 — Session Timeout Notification

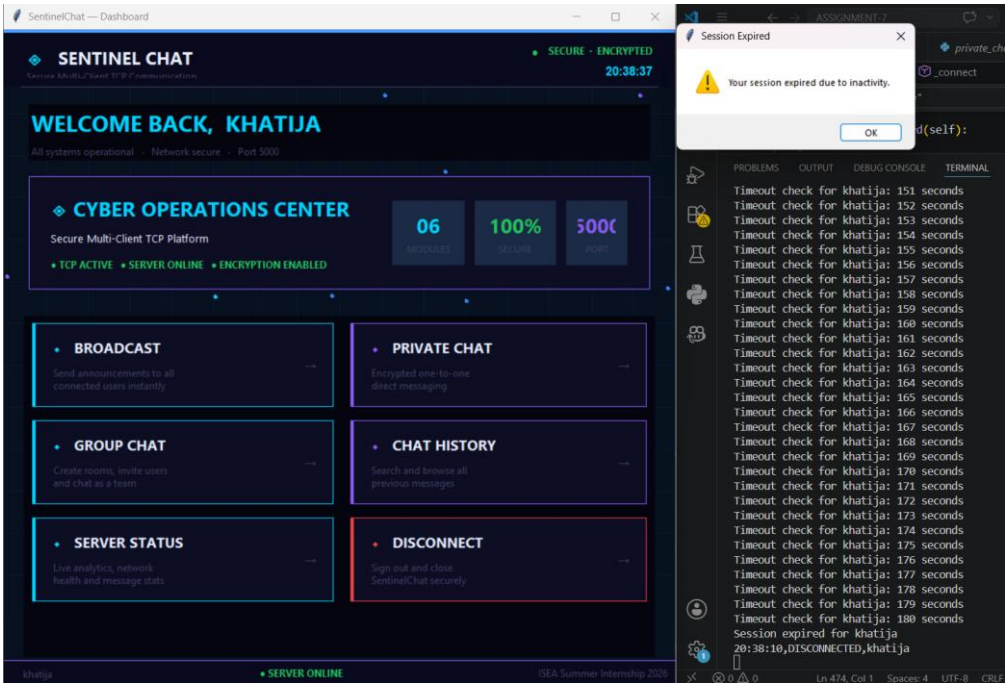


Fig 3.7 — Idle Timeout Countdown

3.6 Secure Logging

Every important security event is recorded inside security_log.txt. The log includes login, logout, failed login, duplicate login, account lock, and session timeout events. Passwords are never written into the log.

```
security_log.txt (excerpt)
[2026-07-16 21:21:05] LOGIN FAILED : admin
[2026-07-16 21:21:10] LOGIN FAILED : admin
[2026-07-16 21:21:16] LOGIN FAILED : admin
[2026-07-16 21:21:19] LOGIN FAILED : admin
[2026-07-16 21:21:23] LOGIN FAILED : admin
[2026-07-16 21:21:23] ACCOUNT LOCKED : admin
[2026-07-16 21:22:01] DUPLICATE LOGIN : admin
[2026-07-16 21:22:24] LOGIN_SUCCESS : khatija
[2026-07-16 21:23:35] SESSION TIMEOUT : admin
[2026-07-16 21:23:35] LOGOUT : admin
```

3.7 Input Validation

The application validates user input before processing requests. Validation includes empty username rejection, empty password rejection, unsupported command rejection, invalid login detection, and oversized message handling. These checks improve application reliability and security.

3.8 Broadcast Messaging

Authenticated users can send broadcast messages to all connected clients simultaneously. Broadcast events are also recorded in the chat history and shown live on the server dashboard.

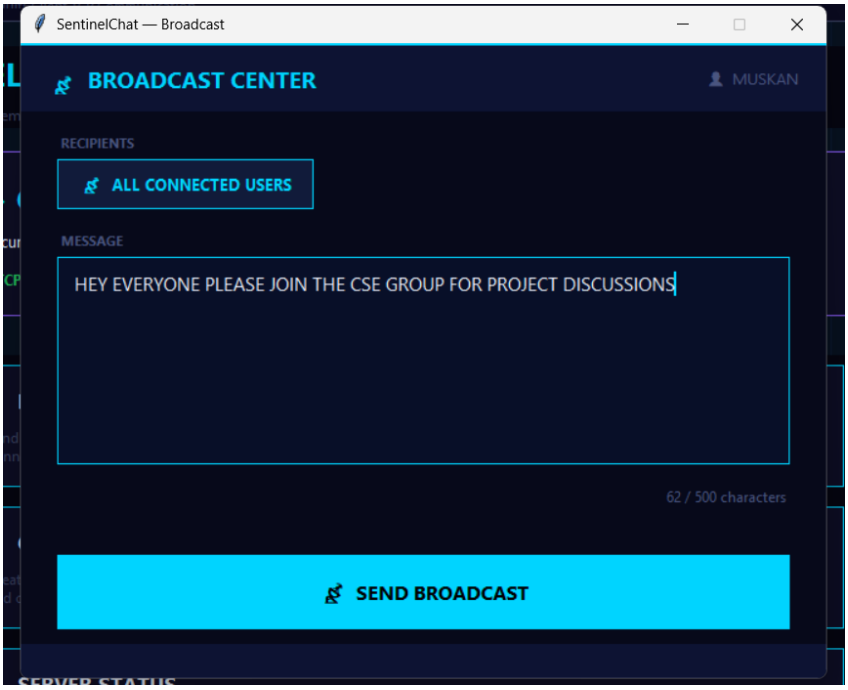


Fig 3.8 — Broadcast Message Sent to All Connected Clients

3.9 Private Messaging

Users can securely communicate with individual users. Private messages are delivered only to the intended recipient and are not visible to other connected clients.

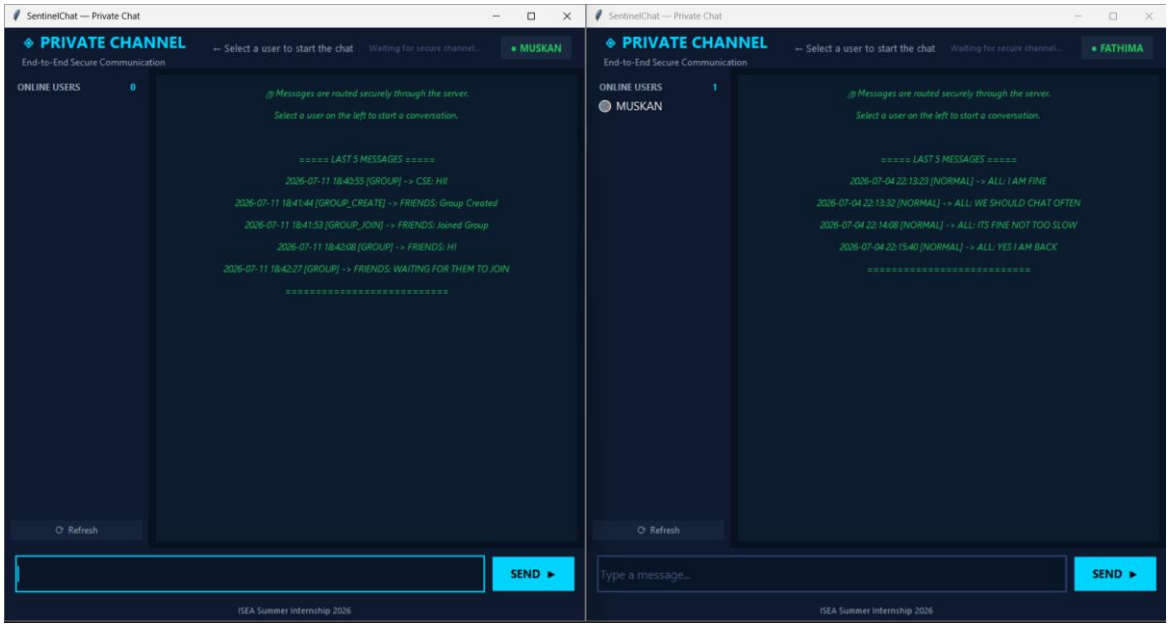


Fig 3.9 — Private One-to-One Chat Window

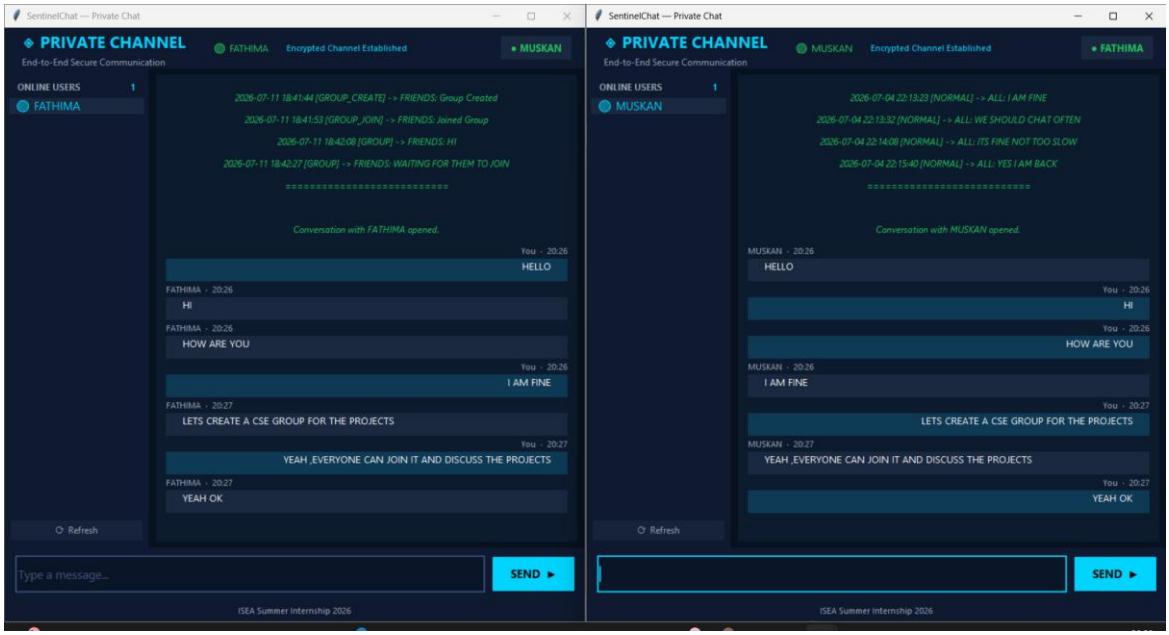


Fig 3.10 — Private Chat, Second Client View

3.10 Group Chat

Users can create groups, join groups, and send messages inside groups. Only members of a group receive the messages sent within it.

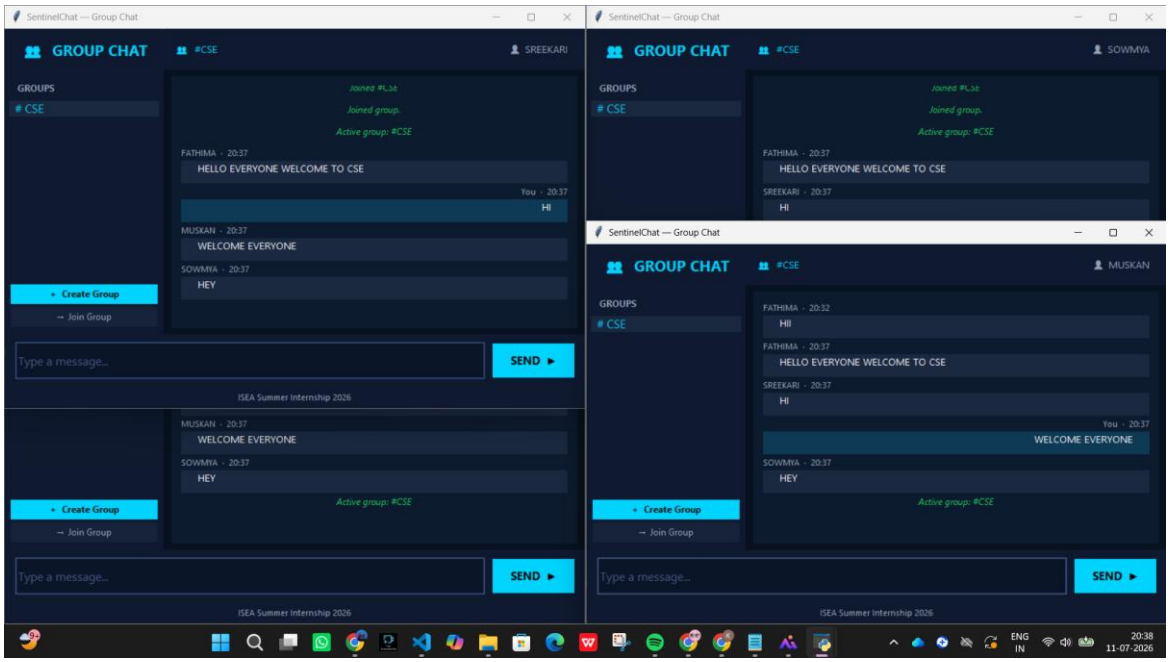


Fig 3.11 — Group Chat Shared Across Multiple Clients

3.11 Chat History

All communication is stored inside chat_history.csv. The history module allows users to search and review previous conversations.

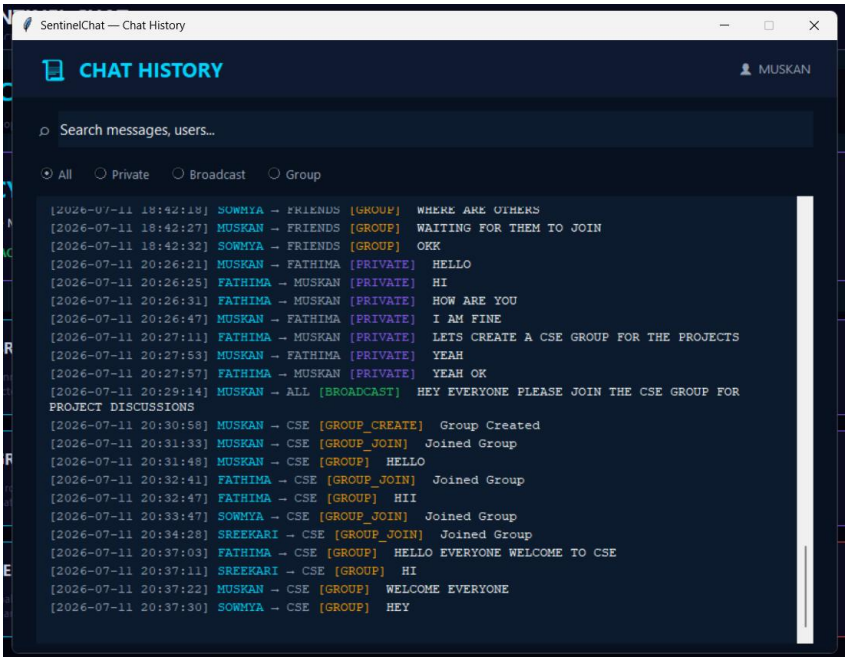


Fig 3.12 — Chat History Search and Review Window

3.12 Server Status Dashboard

A dedicated dashboard displays connected clients, total messages, broadcasts, private messages, CPU/memory throughput, and a live activity log. This helps monitor application health in real time.

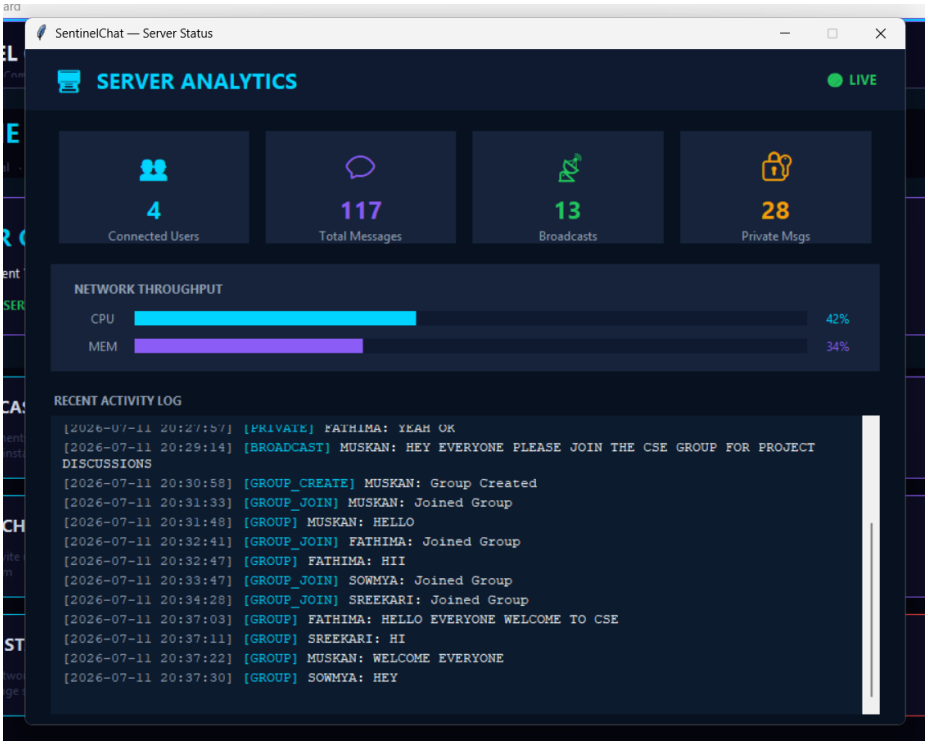


Fig 3.13 — Server Analytics Dashboard

4. System Architecture

The overall architecture consists of a client-server model. Each client connects to the server over a dedicated TCP socket. The server is responsible for authentication, request validation, message routing, activity logging, and session management.

Layer	Component	Responsibility
Client	client_gui.py / dashboard.py	Login UI, dashboard, and feature windows (broadcast, private chat, group chat, history, status)
Transport	TCP Socket (Port 5000)	Reliable, ordered delivery of authentication and chat data between client and server
Server	server.py	Authenticates users, enforces lockouts and timeouts, routes messages, writes logs
Storage	users.json / security_log.txt / chat_history.csv	Persists hashed credentials, security events, and chat history

Client GUI → TCP Socket (Port 5000) → Server → {users.json | security_log.txt | chat_history.csv}

5. Implementation

The project was developed using Python. The graphical interface was built with Tkinter, and TCP socket programming provides communication between clients and the server. The major modules are summarised below.

File	Purpose
server.py	Server implementation, authentication, session and lockout logic
client_gui.py	Login interface
dashboard.py	Main dashboard
private_chat.py	Private messaging
group_chat.py	Group communication
broadcast.py	Broadcast messaging
history.py	Chat history search and display
status.py	Server monitoring dashboard
users.json	User credentials (SHA-256 hashed passwords)
security_log.txt	Security event log
chat_history.csv	Stored conversations

6. Testing

The application was tested using multiple concurrent client instances to verify both functional and security behaviour.

Test Case	Expected Result	Status
Valid login	Login successful	Pass
Invalid password	Login failed	Pass
Duplicate login	Access denied	Pass
Five failed logins	Account locked	Pass
Lock countdown	Countdown displayed	Pass
Login after lock expiry	Successful	Pass
Broadcast message	Delivered to all users	Pass
Private chat	Delivered to recipient only	Pass
Group chat	Delivered to group members only	Pass
Chat history	Saved and searchable	Pass
Session timeout (idle)	Automatic logout	Pass
Secure logging	Events recorded, no passwords logged	Pass

7. Wireshark Verification

Wireshark was used to verify TCP communication on the loopback interface using the filter tcp.port == 5000. The captured traffic confirms the TCP three-way handshake, push/acknowledgement of application data, and connection teardown for every feature exercised.

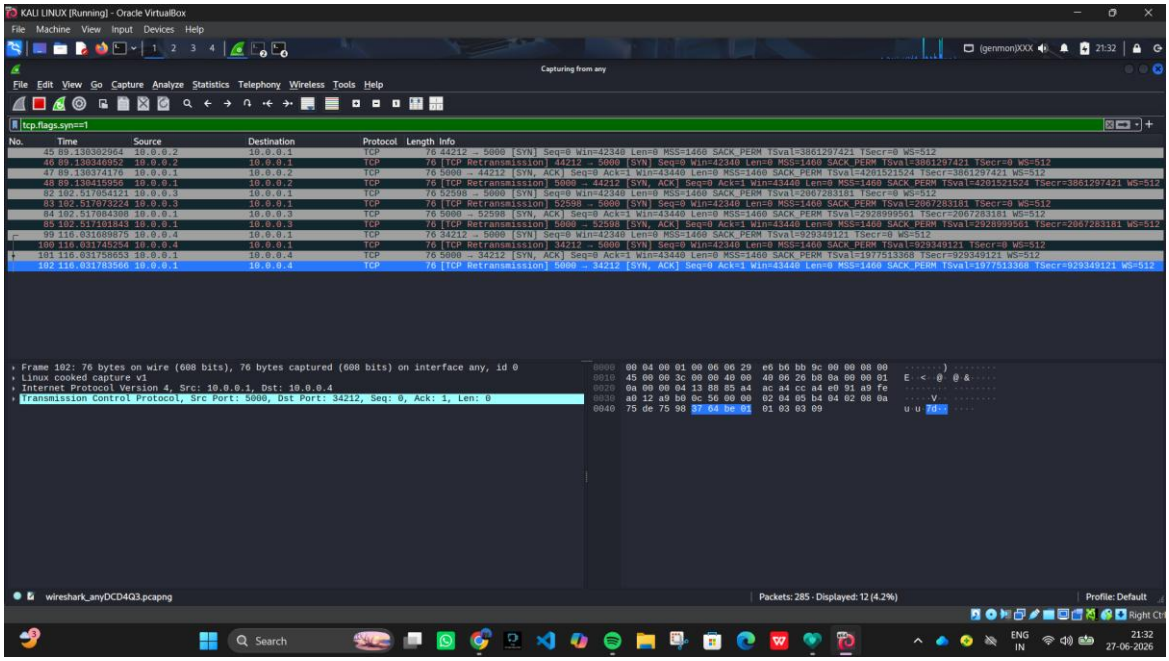


Fig 7.1 — TCP Three-Way Handshake on Port 5000

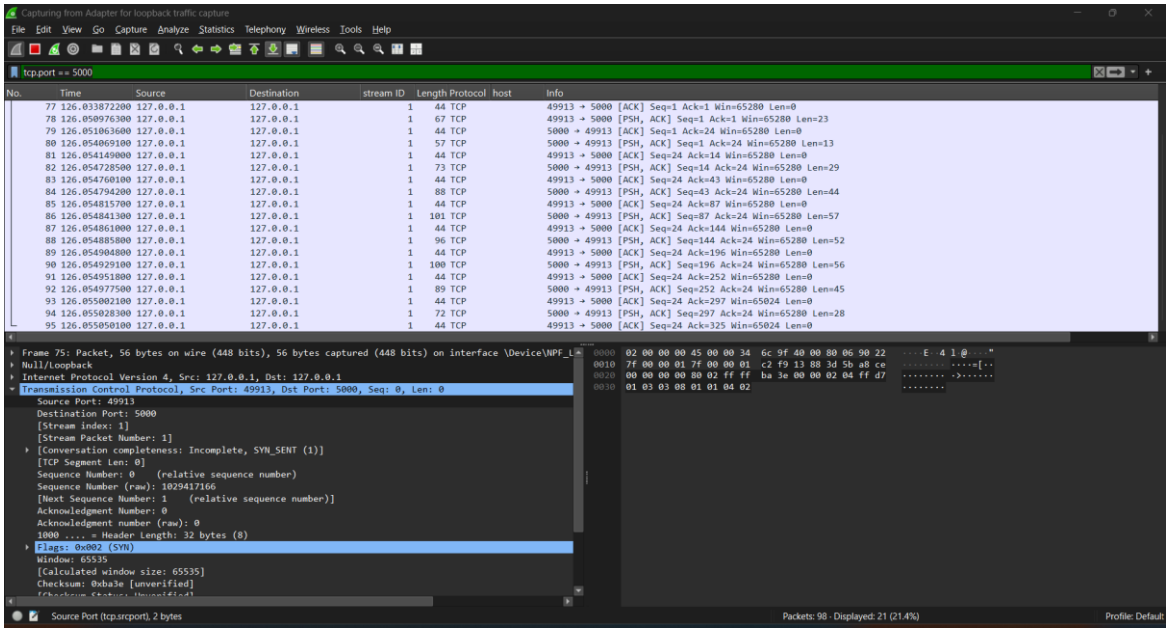


Fig 7.2 — Successful Login Traffic Capture

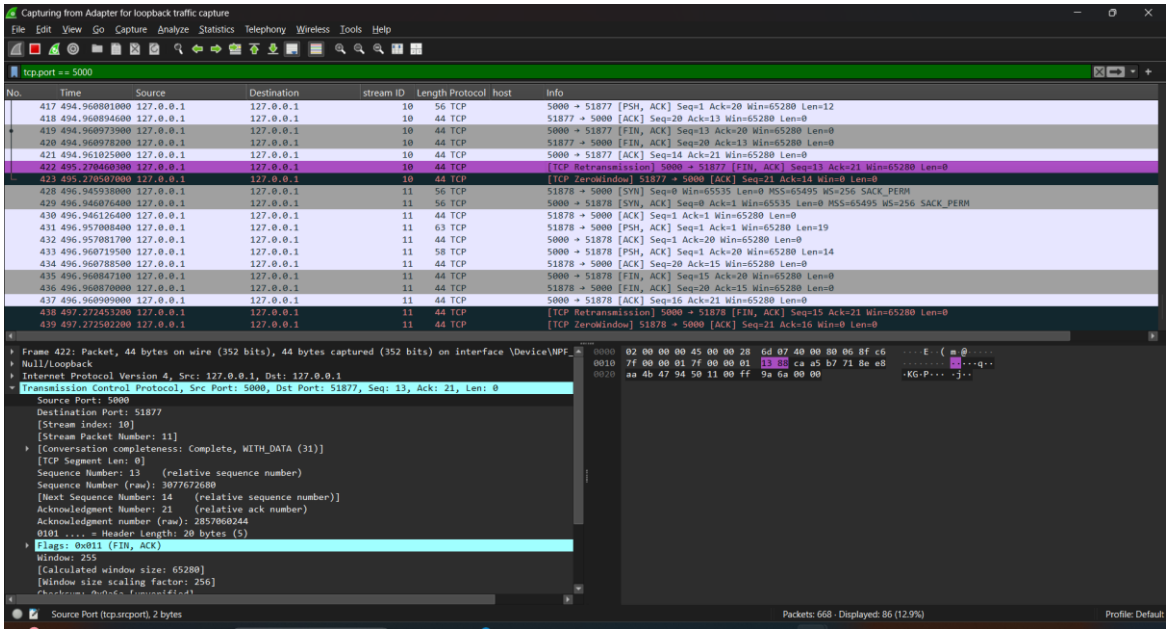


Fig 7.3 — Failed Login Attempt Traffic Capture

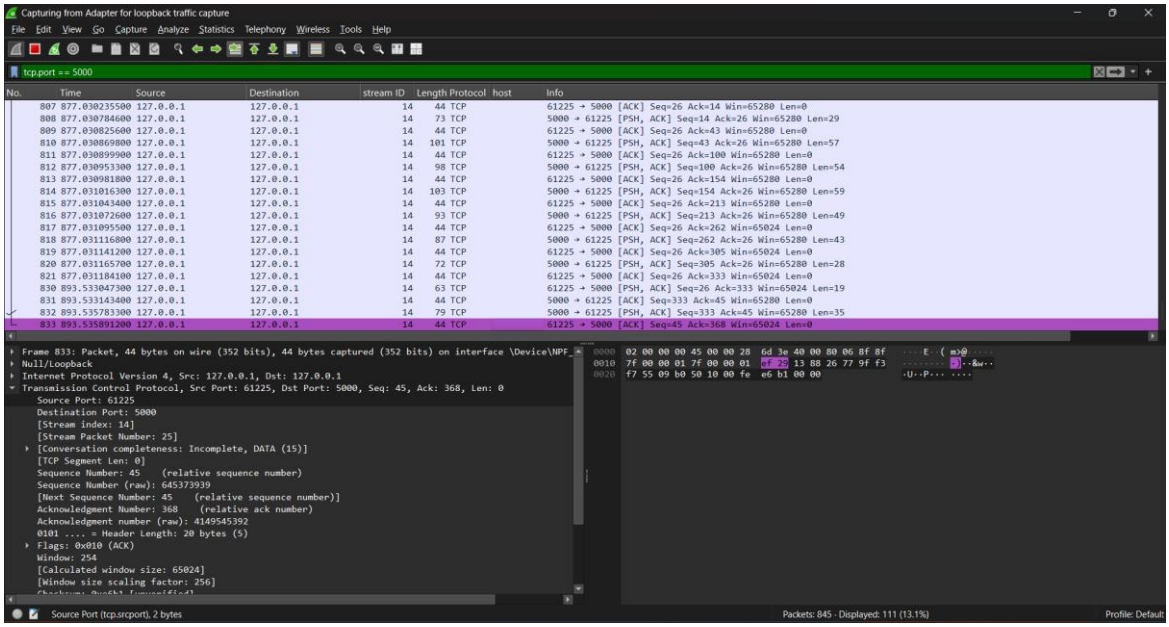


Fig 7.4 — Broadcast Message Traffic Capture

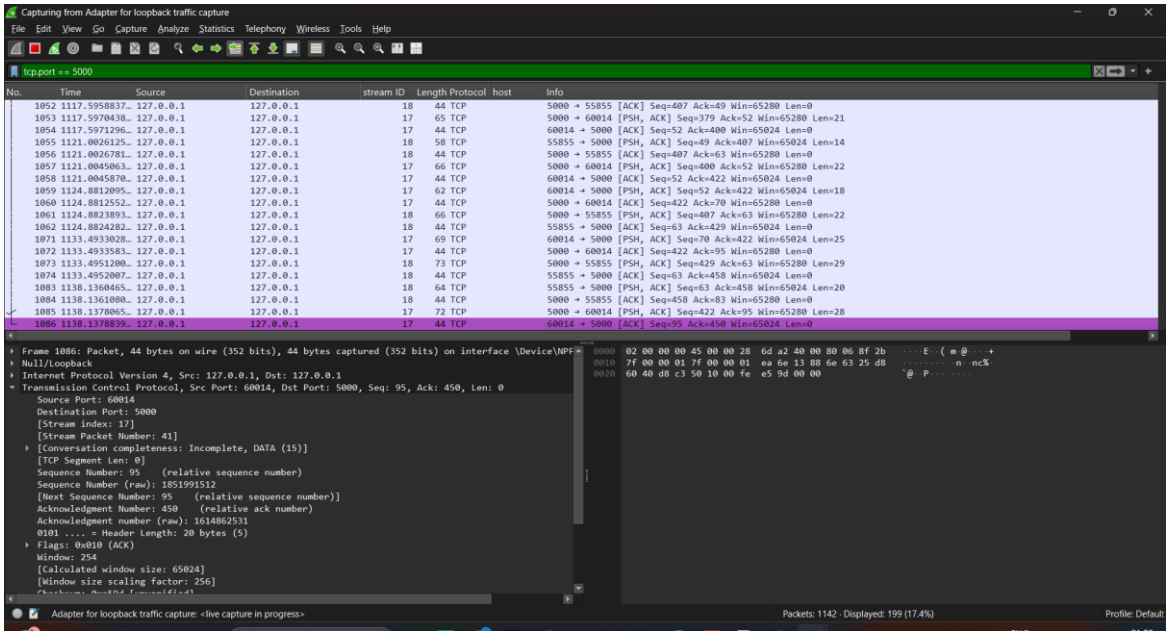


Fig 7.5 — Private Message Traffic Capture

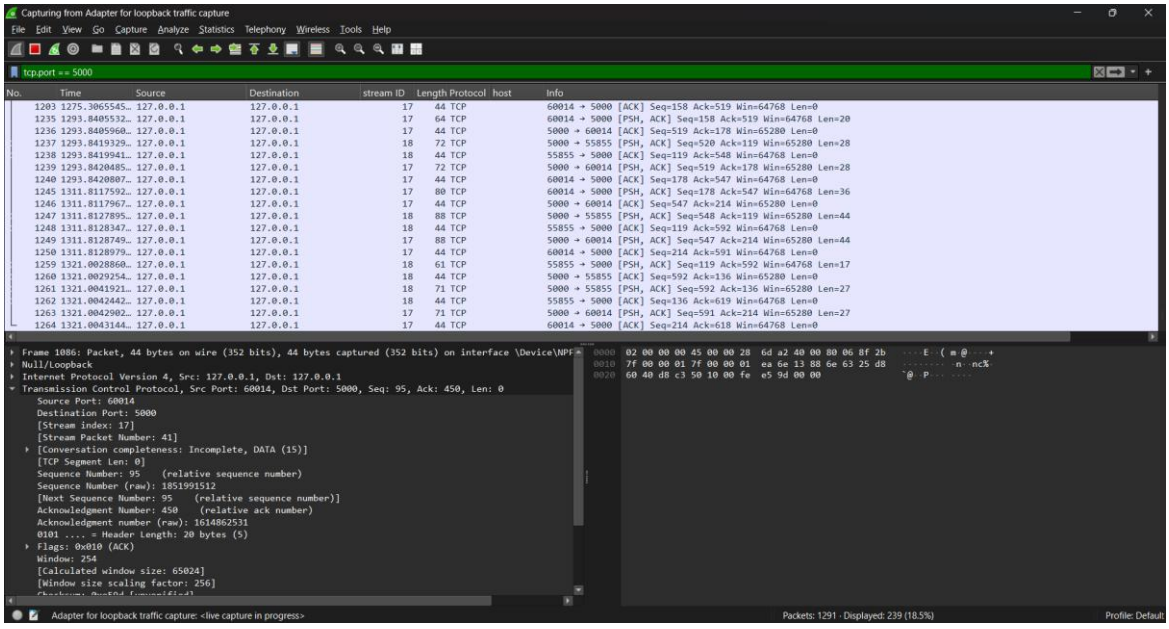


Fig 7.6 — Group Chat Traffic Capture

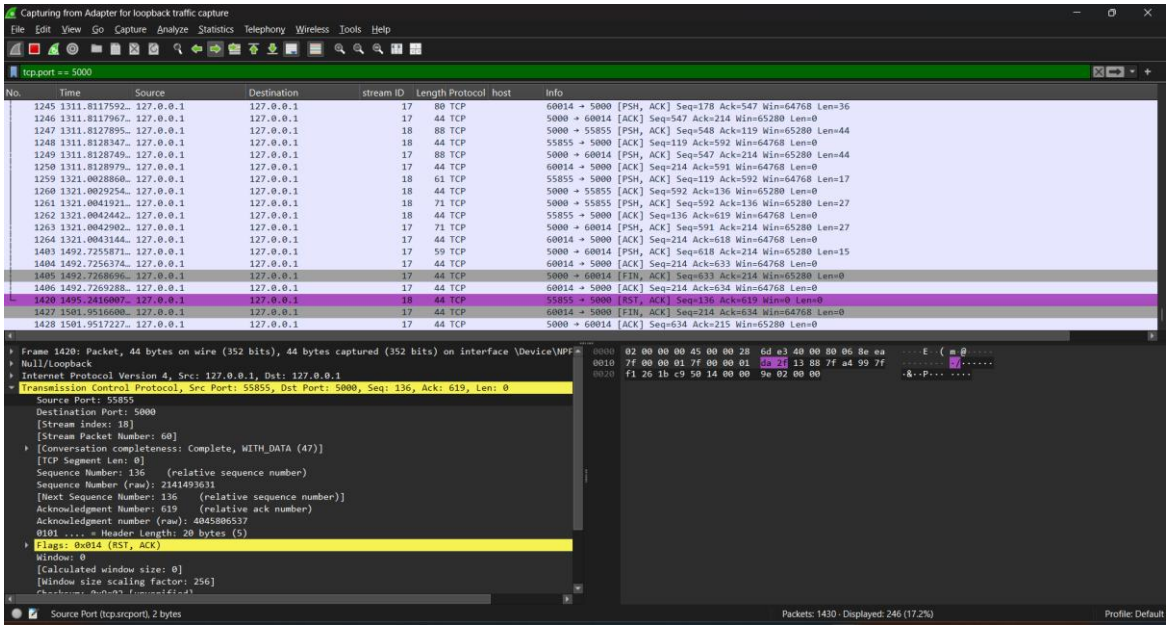


Fig 7.7 — Connection Termination (Disconnect) Capture

Across all captures, communication consistently occurs over TCP port 5000, confirming that every application feature — authentication, broadcast, private messaging, and group chat — is carried over the same secured TCP channel.

8. Conclusion

This assignment successfully transformed a basic TCP chat application into a secure communication platform. Security mechanisms such as password hashing, duplicate login prevention, failed login protection, session timeout, secure logging, and input validation significantly improved the application's reliability and security.

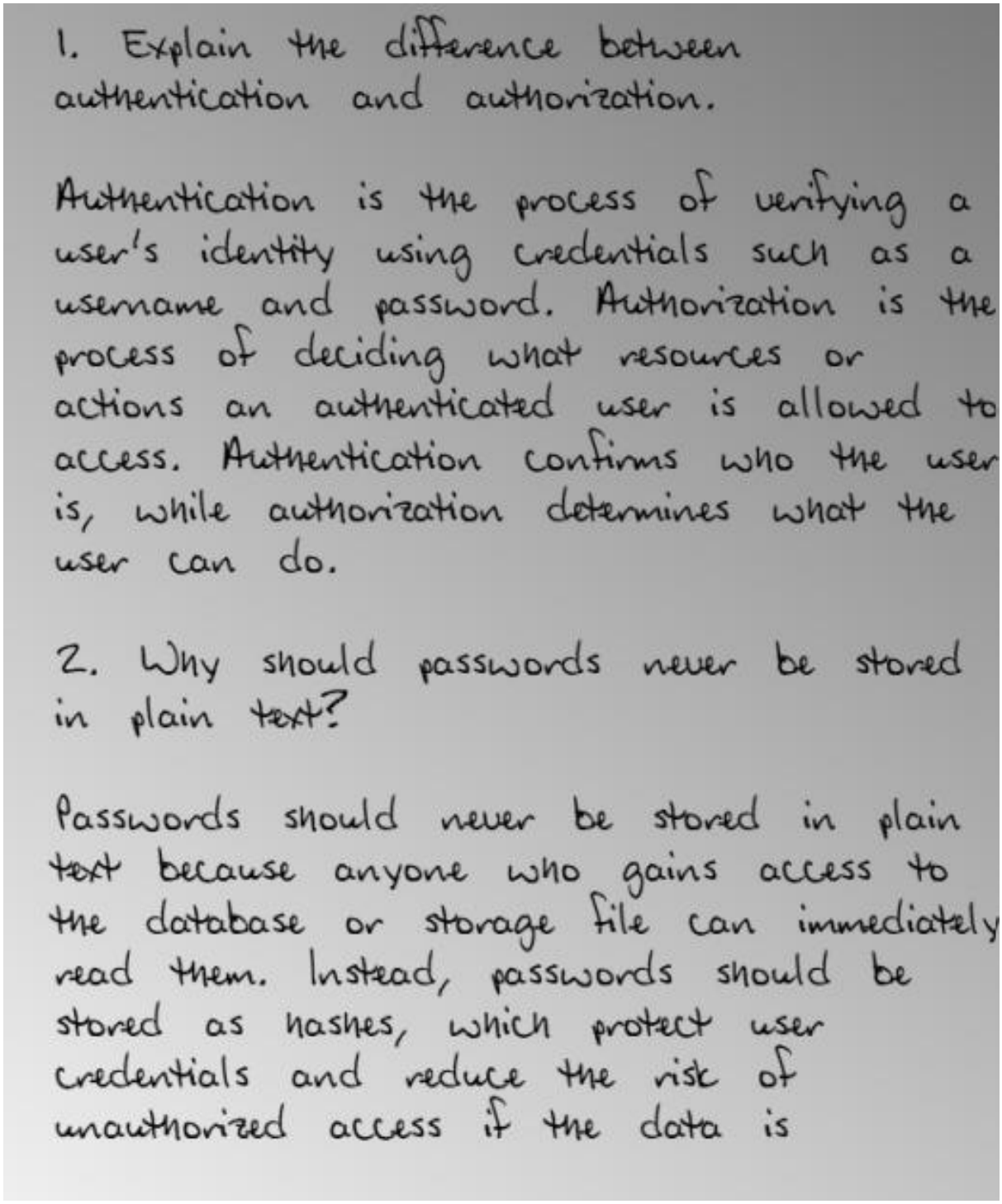
Wireshark verification confirmed successful TCP communication, while extensive testing demonstrated the correct functioning of all implemented features.

The project strengthened practical knowledge of TCP socket programming, authentication mechanisms, secure software development, and network packet analysis. Future improvements may include TLS encryption, database integration, role-based access control, and two-factor authentication.

9. References

- Python Official Documentation — https://docs.python.org
- Python Socket Programming Documentation
- Python hashlib Documentation
- Wireshark User Guide — https://www.wireshark.org/docs/
- ISEA Summer Internship Assignment 7 Specification

REFLECTION QUESTIONS



1. Explain the difference between authentication and authorization.

Authentication is the process of verifying a user's identity using credentials such as a username and password. Authorization is the process of deciding what resources or actions an authenticated user is allowed to access. Authentication confirms who the user is, while authorization determines what the user can do.

2. Why should passwords never be stored in plain ~~text~~?

Passwords should never be stored in plain text because anyone who gains access to the database or storage file can immediately read them. Instead, passwords should be stored as hashes, which protect user credentials and reduce the risk of unauthorized access if the data is

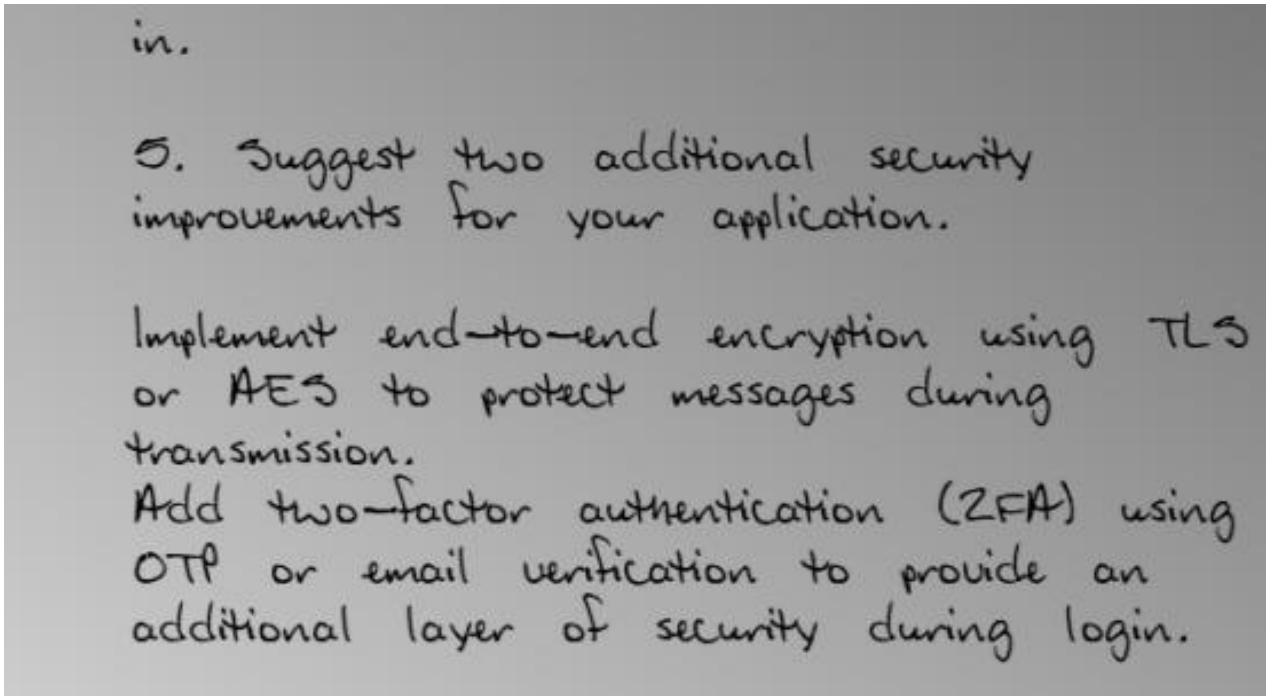
compromised.

3. What is the purpose of password hashing?

Password hashing converts a password into a unique fixed-length hash value using a one-way algorithm such as SHA-256. During login, the entered password is hashed and compared with the stored hash instead of storing or comparing the actual password. This improves the security of user accounts.

4. How does duplicate login prevention improve security?

Duplicate login prevention allows only one active session for a user account at a time. It prevents unauthorized users from accessing the same account simultaneously, reduces the risk of session hijacking, and improves overall account security by ensuring only the legitimate user remains logged


in.
5. Suggest two additional security improvements for your application.
Implement end-to-end encryption using TLS or AES to protect messages during transmission.
Add two-factor authentication (2FA) using OTP or email verification to provide an additional layer of security during login.